

Gated Slot Attention for Efficient Linear-Time Sequence Modeling

Yu Zhang^{1*} Songlin Yang^{2*} Ruijie Zhu³ Yue Zhang¹ Leyang Cui⁴
 Yiqiao Wang⁵ Bolun Wang⁵ Freda Shi⁶ Bailin Wang²
 Wei Bi⁴ Peng Zhou^{5†} Guohong Fu^{1†}

¹School of Computer Science and Technology, Soochow University, China

²Massachusetts Institute of Technology ³University of California, Santa Cruz

⁴Tencent AI Lab ⁵LuxiTech ⁶University of Waterloo

yzhang.cs@outlook.com yangsl66@mit.edu

🔗 <https://github.com/sustcsonglin/flash-linear-attention>

🤗 <https://huggingface.co/fla-hub>

Abstract

Linear attention Transformers and their gated variants, celebrated for enabling parallel training and efficient recurrent inference, still fall short in recall-intensive tasks compared to traditional Transformers and demand significant resources for training from scratch. This paper introduces Gated Slot Attention (GSA), which enhances Attention with Bounded-memory-Control (ABC [63]) by incorporating a gating mechanism inspired by Gated Linear Attention (GLA [96]). Essentially, GSA comprises a two-layer GLA linked via softmax, utilizing context-aware memory reading and adaptive forgetting to improve memory capacity while maintaining compact recurrent state size. This design greatly enhances both training and inference efficiency through GLA’s hardware-efficient training algorithm and reduced state size. Additionally, retaining the softmax operation is particularly beneficial in “finetuning pretrained Transformers to RNNs” (T2R [41]) settings, reducing the need for extensive training from scratch. Extensive experiments confirm GSA’s superior performance in scenarios requiring in-context recall and in T2R settings.

1 Introduction

Transformers [88] have emerged as the predominant architecture for *most, if not all*, sequence modeling tasks. Nevertheless, the quadratic complexity of softmax-based standard attention (SA) poses significant challenges for long sequence modeling (e.g., video understanding and biological sequence modeling). In the context of language modeling, where sequence lengths are moderate, training efficiency is generally not a primary concern. However, during inference, the Key-Value (KV) cache [34, 64] grows linearly with the generation length, resulting in substantial memory burdens and throughput bottlenecks due to high I/O costs.

Linear (kernelized) attention [42] and its gated variants [96, 82, 68, 61, 16, 69] have received interest as promising alternatives to softmax attention. These models demonstrate strong performance in language modeling and understanding tasks. Notably, they can be reframed as RNNs during inference, achieving constant memory complexity and thereby significantly enhancing inference efficiency.

However, two key issues persist with these models: (i) Performance-wise, recent research indicates that linear recurrent models still struggle with tasks requiring in-context retrieval or learning [2, 1, 37,

*Equal contributions. Work was conducted during Yu Zhang’s internship at Tencent AI Lab.

†Corresponding authors.

28], and there is a *fundamental* recall-memory trade-off [3, 91] where all inference-time-constant-memory models face inherent limitations. (ii) In terms of training efficiency, while linear attention supports hardware-efficient chunkwise training [96] as implemented in FlashLinearAttention (FLA [95]), training from scratch on trillions of tokens remains prohibitively expensive. A paradigm, “*finetuning pretrained Transformers to RNNs*” (short for T2R [41]), has recently gained great attention [101, 10, 54, 13, 7, 90]. This approach circumvents the high cost of training from scratch by requiring only a few billion tokens for finetuning—about 1–3% of the total cost. However, linear attention uses a different kernel method from softmax, leading to performance discrepancies when finetuning pretrained softmax attention models to linear attention [101].

To address these issues, we revisit the Attention with Bounded-Memory Control (ABC) model [63], which retains the softmax operation, thereby reducing training-finetuning discrepancies between standard and linear attention, making it ideal for T2R settings. Additionally, ABC enables more effective state utilization, requiring less state size to achieve similar performance, as observed in Peng et al. [63]. This results in more efficient inference and potentially expands the Pareto frontier of the recall-memory tradeoff [3]. However, ABC has not gained significant attention due to its mediocre language modeling performance and slow training speed.

In this work, we first reformulate ABC as two-pass linear attention linked via softmax, allowing us to leverage the hardware-efficient chunkwise implementation from FLA [95] for more efficient training. We then identify several limitations of ABC and propose a new model, dubbed Gated Slot Attention (GSA), which is essentially a gated version of ABC, following the recent trend of enhancing linear attention with gating mechanisms [96, 69, 61, 16, 6, 62, 52, 65].

Our extensive evaluation shows that GSA not only matches performance in language modeling and understanding tasks but also significantly outperforms other linear models in *in-context recall-intensive* tasks [3, 4], without requiring a large state size like RetNet [82] or GLA [96]. In the T2R finetuning setting, we found that finetuning Mistral-7B [39] to GSA surpasses large recurrent language models (e.g., RWKV6-7B, Mamba-7B) and also outperforms finetuning Mistral-7B to other linear models (e.g., RetNet, GLA) and other T2R methods like SUPRA [54], verifying the importance of retaining the softmax operator. Finally, we remark that GSA achieves similar training speeds to GLA while offering an inference speedup due to its smaller state size.

2 Background and Preliminary

2.1 Transformers as Unbounded Key-Value Memories

Given $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_T]^\top \in \mathbb{R}^{T \times d}$, where T is the sequence length and $\mathbf{x}_i \in \mathbb{R}^d$ is the i -th input vector with d dimensions, SA with causal masking computes the output matrix:

$$\mathbf{O} = f((\mathbf{Q}\mathbf{K}^\top) \odot \mathbf{M})\mathbf{V}, \quad (1)$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times d}$ are linear mappings of the input $\mathbf{X}$ via learnable weights $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{d \times d}$, $\mathbf{M} = \{M_{ij} = 1 \text{ if } i \geq j \text{ o.w. } -\infty\}$ is the causal mask to prevent future information leakage, $\odot$ denotes element-wise production, and $f(\cdot)$ is $\text{softmax}(\cdot)$.

Generally, $\mathbf{K}, \mathbf{V}$ can be viewed as neural *key-value memories* $\tilde{\mathbf{K}}_t, \tilde{\mathbf{V}}_t \in \mathbb{R}^{m \times d}$, respectively [81, 24], where m is the number of memory slots. At step t , the query $\mathbf{q}_t = \mathbf{W}_q \mathbf{x}_t \in \mathbb{R}^d$ first attends to the key memories $\tilde{\mathbf{K}}_t$ to retrieve relevant information, which is then summarized into $\mathbf{o}_t$ by computing a weighted sum of the value memories $\tilde{\mathbf{V}}_t$ [104], where the weights are the normalized attention scores:

$$\mathbf{o}_t = \tilde{\mathbf{V}}_t^\top f(\tilde{\mathbf{K}}_t \mathbf{q}_t). \quad (2)$$

From this perspective, Transformers are equipped with an unbounded number of memory slots, which grow linearly with respect to the sequence length [57] (i.e., $m = t$ for step t)—a new key $\mathbf{k}_t = \mathbf{W}_k \mathbf{x}_t \in \mathbb{R}^d$ is assigned with a unique memory slot upon its introduction. This leads to a simple memory updating rule: $\tilde{\mathbf{K}}_t = \tilde{\mathbf{K}}_{t-1} \cup \{\mathbf{k}_t\}$. The value memories $\tilde{\mathbf{V}}_t$ are updated in a similar way. This mechanism, however, comes at the cost of quadratic time complexity in terms of the sequence length for training and $O(Td)$ time/memory complexity for inference [64], posing challenges for large-scale models.

2.2 ABC [63]: Linearizing Attention with Bounded Memory Control

From a key-value memory perspective, the training and inference complexity of self-attention (SA) can be reduced by fixing the number of memory slots to a constant size $m \ll T$ [27, 51, 63]. One straightforward way to achieve this is by employing a *first-in-first-out* memory management strategy, commonly known as sliding window attention (SWA). However, SWA is inefficient because it discards all information outside the window, leading to poor performance in balancing the recall-memory tradeoff [3]. To achieve acceptable performance, SWA often requires a large window size (e.g., 4,096 tokens in Mistral [39]), which diminishes its advantage over to global attention.

When the number of tokens in a sequence exceeds the number of memory slots, it becomes necessary to store information from multiple tokens in a single slot. To address this challenge, Peng et al. [63] propose the Attention-with-Bounded-memory-Control (ABC) mechanism, which allows multiple tokens to be written into a single slot:

$$\tilde{\mathbf{K}}_t = \tilde{\mathbf{K}}_{t-1} + \phi_t \otimes \mathbf{k}_t \in \mathbb{R}^{m \times d}, \quad \tilde{\mathbf{V}}_t = \tilde{\mathbf{V}}_{t-1} + \phi_t \otimes \mathbf{v}_t \in \mathbb{R}^{m \times d}, \quad \mathbf{o}_t = \tilde{\mathbf{V}}^T f(\tilde{\mathbf{K}}_t^T \mathbf{q}_t) \in \mathbb{R}^d \quad (3)$$

where

$$\alpha_i = \exp(\mathbf{W}_\phi \mathbf{x}_i) \in \mathbb{R}^m, \quad \phi_i = \frac{\alpha_i}{\sum_{j=1}^i \alpha_j} \in (0, 1)^m \quad (4)$$

Here, $(\phi_i)_j$ represents the writing intensity of the i th token to the j th slot, obtained using a cumulative softmax function (cf. [63, footnote 5]), which can be computed with a prefix sum.

ABC as two-pass linear attention. The outer-product-based additive memory update rule in Eq. 3 bears a resemblance to linear attention [42], which involves the following recurrence³:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t \otimes \mathbf{v}_t \in \mathbb{R}^{d \times d}, \quad \mathbf{o}_t = \mathbf{S}_t^T \mathbf{q}_t \in \mathbb{R}^d \quad (5)$$

We denote this linear attention operator that computes $\mathbf{o}_i$ from $\mathbf{q}_i$, $\mathbf{k}_i$ and $\mathbf{v}_i$ (Eq. 5) by $\{\mathbf{o}_i\}_{i=1}^T = \text{LA}(\{\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i\}_{i=1}^T)$. We show that the ABC operations can be written as

$$\begin{aligned} \{\mathbf{o}'_i\}_{i=1}^T &= \text{LA}(\{\mathbf{q}_i, \mathbf{k}_i, \phi_i\}_{i=1}^T), \\ \{\mathbf{o}_i\}_{i=1}^T &= \text{LA}(\{\text{softmax}(\mathbf{o}'_i), \phi_i, \mathbf{v}_i\}_{i=1}^T), \end{aligned}$$

where $\mathbf{o}'_i \in \mathbb{R}^m$, $\mathbf{o}_i \in \mathbb{R}^d$. Therefore, ABC can enjoy hardware-efficient linear-time chunkwise training [96], as implemented in the FLA library [95].

Remarks on state size. Peng et al. [63] empirically demonstrated that ABC requires a smaller state size to achieve comparable performance to other linear attention models, resulting in improved inference efficiency. We offer the following intuitive explanation: the new query $\mathbf{o}'$ aggregates the entire history through the initial pass of linear attention, making it more context-aware and better at locating desired items for retrieval. The subsequent softmax operator helps mitigate the attention dilution issue [66]. From the perspective of Hopfield networks, softmax can exponentially increase the memory size [45]. Together, these factors suggest that ABC may possess an implicit large memory capacity, even with a small actual recurrent state size.

2.3 GLA [96]: Linear Attention with Gating Mechanism

Linear attentions underperform softmax-attention Transformers in language modeling by a notable margin. RetNet [82] and TransormerLLM [68] incorporate a *data-independent exponential decay* factor for memory update as

$$\mathbf{S}_t = \gamma \mathbf{S}_{t-1} + \mathbf{k}_t \otimes \mathbf{v}_t \in \mathbb{R}^{d \times d},$$

where $\gamma \in (0, 1)$ is a scalar data-independent decaying factor; that is, the decay rate is fixed across time steps and hidden channels (under the same head), disrespects to the input tokens. RetNet has shown better language modeling performance compared to vanilla linear attentions thanks to the decaying mechanism.

³For simplicity, we omit the normalization term, which has been shown to be unnecessary [75, 66, 52, 82, 96].

However, research in recurrent neural networks (RNNs) has shown that *data-dependent* decay (or forget gates) is crucial for selectively retaining and forgetting information [22, 26], thus better leveraging the fixed recurrent hidden state. This selective mechanism has been revisited in recent state-space models [29, 16]. Inspired by LSTMs, Gated Linear Attention (GLA) [52, 96] introduces data-dependent decay parameters $\mathbf{G}_t \in (0, 1)^{d \times d}$ to gate the hidden state as follows,

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t \otimes \mathbf{v}_t \in \mathbb{R}^{d \times d}, \quad \mathbf{o}_t = \mathbf{S}_t^T \mathbf{q}_t \in \mathbb{R}^d.$$

[96] show that if gates are parameterized in an outer product form $\mathbf{G}_t = \boldsymbol{\alpha}_t \otimes \boldsymbol{\beta}_t$, and $\boldsymbol{\alpha}_t, \boldsymbol{\beta}_t \in [0, 1]^d$ depend solely on input $\mathbf{x}_t$, such recurrence can be rewritten as matrix multiplication, allowing for hardware-efficient training with a chunkwise parallel form. In what follows, we will use the following notation $\text{GLA}(\{\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i, \boldsymbol{\alpha}_i, \boldsymbol{\beta}_i\}_{i=1}^T) = \{\mathbf{o}_i\}_{i=1}^T$ to denote this computation. It is common to set $\beta_i = \mathbf{1}$ as in [96, 69, 61], which is also often written in the following equivalent form:

$$\mathbf{S}_t = \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1} + \mathbf{k}_t \otimes \mathbf{v}_t.$$

Here $\mathbf{k}_t$ can be viewed as the input gate, and $\boldsymbol{\alpha}_t$ can be viewed as the forget gate. In gated RNN literature, it is common to couple these two gates via $\mathbf{k}_t = \mathbf{1} - \boldsymbol{\alpha}_t$ [12, 106, 67]. In particular, Qin et al. [69] proposed HGRN2, which uses this strategy as an improved parameterization of GLA, showing better performance in language modeling.

3 Method

3.1 Motivation: Issues with ABC

We identify two primary limitations in ABC’s memory update rule. Firstly, it **lacks a forgetting mechanism**, resulting in indefinite retention of items once written into memory slots. This prevents efficient memory reuse by impeding the prompt clearance of slots for new information.

Secondly, **the rule introduces an unwarranted inductive bias favoring tokens at the sentence’s beginning. This contradicts the recency bias in natural language, where more recent information is often more relevant.** Prioritizing initial tokens over the recent ones conflicts with this inherent tendency in natural language processing.

Specifically, for the first token, the writing strength to all slots is maximized (i.e., $\phi_1 = \mathbf{1} \in \mathbb{R}^m$), causing every memory slot to retain a copy of the first token’s representation. The absence of a forgetting mechanism exacerbates this issue. For subsequent tokens, the writing strength diminishes due to the influence of earlier tokens, as a result of the cumulative softmax in Eq. 4. This makes it challenging for the model to retain later tokens without learning a significantly large α_i , potentially leading to instability in long-context settings, as observed by Zhang et al. [100].

3.2 Gated Slot Attention (GSA): ABC with gating mechanism

To address these limitations, we propose Gated Slot Attention (GSA), which incorporates a gating mechanism to simultaneously resolve both issues by: **(i) enabling the forgetting** of historical information, and **(ii) introducing a recency inductive bias**, as detailed below.

For each memory slot, the update rule is a simple gated RNN with a scalar data-dependent gating value $\alpha_i \in [0, 1]$,

$$(\tilde{\mathbf{K}}_t)_i = \alpha_i (\tilde{\mathbf{K}}_{t-1})_i + (1 - \alpha_i) \mathbf{k}_t \in \mathbb{R}^d, \quad (\tilde{\mathbf{V}}_t)_i = \alpha_i (\tilde{\mathbf{V}}_{t-1})_i + (1 - \alpha_i) \mathbf{v}_t \in \mathbb{R}^d$$

and these can be written in matrix form, which is reminiscent of HGRN2 [69].

$$\begin{aligned} \tilde{\mathbf{K}}_t &= \text{Diag}(\boldsymbol{\alpha}_t) \cdot \tilde{\mathbf{K}}_{t-1} + (1 - \boldsymbol{\alpha}_t) \otimes \mathbf{k}_t \in \mathbb{R}^{m \times d} \\ \tilde{\mathbf{V}}_t &= \text{Diag}(\boldsymbol{\alpha}_t) \cdot \tilde{\mathbf{V}}_{t-1} + (1 - \boldsymbol{\alpha}_t) \otimes \mathbf{v}_t \in \mathbb{R}^{m \times d} \\ \mathbf{o}_t &= \tilde{\mathbf{V}}^T \text{softmax}(\tilde{\mathbf{K}}_t^T \mathbf{q}_t) \in \mathbb{R}^d \end{aligned} \tag{6}$$

GSA as two-pass GLA. It is straightforward to see that we can write GSA as a two-pass GLA as shown below:

$$\begin{aligned} \{\mathbf{o}'_t\}_{t=1}^T &= \text{GLA}(\{\mathbf{q}_t, \mathbf{k}_t, 1 - \boldsymbol{\alpha}_t, \boldsymbol{\alpha}_t, \mathbf{1}\}_{t=1}^T) \\ \{\mathbf{o}_t\}_{t=1}^T &= \text{GLA}(\{\text{softmax}(\mathbf{o}'_t), 1 - \boldsymbol{\alpha}_t, \mathbf{v}_t, \mathbf{1}, \boldsymbol{\alpha}_t\}_{t=1}^T) \end{aligned} \tag{7}$$

Therefore, we can adapt GLA’s hardware-efficient chunkwise training algorithm for GSA training, as shown in § A and § B. We illustrate the recurrent representation of GSA in Figure 1.

3.3 Neural Architecture

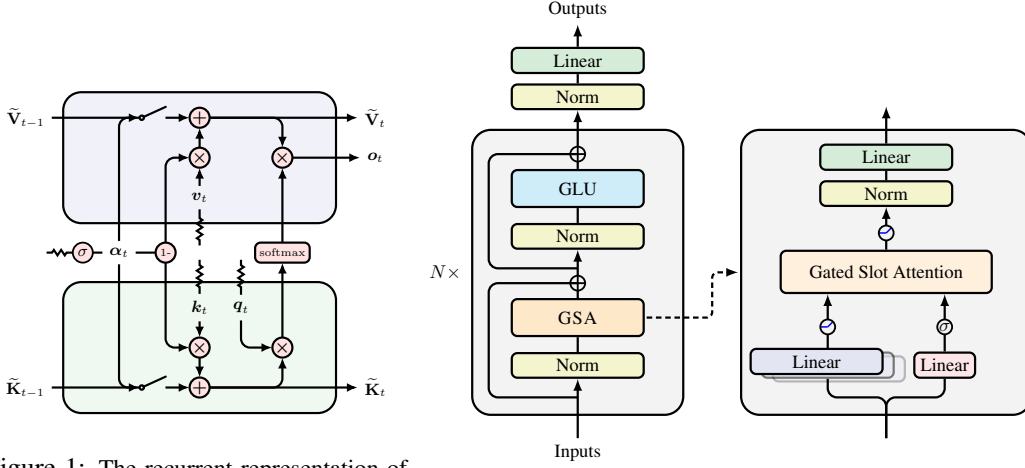


Figure 1: The recurrent representation of GSA. $\sim$ means taking x_t as input.

Figure 2: The backbone of our proposed GSA models.

The overall architecture of our proposed model, GSA, is shown in Figure 2. Following the Llama architecture [86], we use a stack of L GSA blocks, each comprising a GSA token mixing layer followed by a Gated Linear Unit (GLU) channel mixing layer [19, 33].

We utilize the multi-head attention mechanism [88] to capture different aspects of the input. For each head h , the input to GSA token mixing is defined as

$$q_i^h, k_i^h, v_i^h = \phi(\mathbf{W}_q^h x_i), \phi(\mathbf{W}_k^h x_i), \phi(\mathbf{W}_v^h x_i) \quad (8)$$

where ϕ is the Swish activation following [68]. The forget gate is obtained by a linear transformation followed by a **sigmoid activation σ with a damping factor τ** [96, 83]: $\alpha_i^h = \sigma(\mathbf{W}_\alpha^h x_i)^{1/\tau}$,⁴ where **the damping factor is to regulate the forget gate value to one, which has been shown to be crucial for long-term dependency modeling** [30, 67]. We feed them into a GSA layer to obtain outputs as described in Eq. 7:

$$\{o_i^h\}_{i=1}^T = \text{GSA}(\{q_i^h, k_i^h, v_i^h, \alpha_i^h\}_{i=1}^T)$$

Finally, we obtain output via

$$y_i = \mathbf{W}_o (\text{RMSNorm} (\text{Swish} (\text{Concat} (o_i^1, \dots, o_i^H)))) \quad (9)$$

The total number of parameters for $\mathbf{W}_q$, $\mathbf{W}_k$, $\mathbf{W}_v$, and $\mathbf{W}_o$ is already $4d^2$, which is the same as in a single standard softmax-attention layer. To control the overall parameter count, we aim to keep the parameters for $\mathbf{W}_\alpha$, which amount to dHm , relatively small. In practice, we set $m = 64$ to achieve a balance between efficiency and effectiveness (§ 4.1.4). One way to further manage the total parameter count is by reducing the number of heads. In practice, we set $H = 4$, ensuring that $Hm \ll d$. This keeps the total number of parameters approximately equal to $4d^2$.⁵

4 Experiments

4.1 Language Modeling

We perform moderate-scale language modeling experiments with 1.3B and 2.7B parameters on Slimpajama corpus [79] for 100B tokens each.

We compare the performance of GSA against Llama Transformer architecture (i.e., Xfmr++ [86] and recent subquadratic architectures including: Mamba [29], RetNet [82], GLA [96] and HGRN2 [69]. We refer readers to § C for more details on baselines and other experimental setups.

⁴In practice we set $\tau = 8$.

⁵For instance, in a 1.3B model with $H \times m = 64 \times 4 = 256$ and $d = 2,048$, the total number of parameters amount to $4.125d^2$, introducing only a $0.125d^2$ overhead.

4.1.1 Results on commonsense reasoning tasks

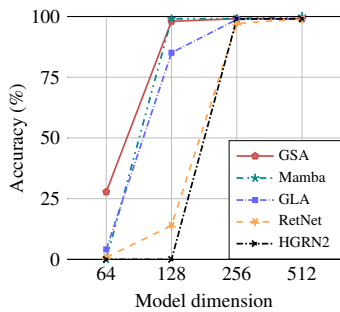
Following [29, 96], we report the perplexities and zero-shot performance of commonsense reasoning tasks including ARC_e & ARC_c (ARC-easy, ARC-challenge) [14]; Hella. (Hellaswag) [99], Lamb. (Lambada) [59], PIQA [8], Wiki. (Wikitext) [55], and Wino. (Winograde) [73]. We note that these tasks are typically short in length and do not require in-context learning capabilities, thus they do not adequately reflect long-context modeling or in-context learning retrieval abilities. Nevertheless, as shown in Table 1, we found that GSA performs comparably to the recent strong model HGRN2 with an equally sized hidden state, while outperforming GLA and RetNet even with a smaller state size.

Table 1: The zero-shot results of 1.3B and 2.7B models evaluated by lm-evaluation-harness [21]. L denotes number of layer while d denotes the model dimension.

	State size	Lamb. ppl _↓	Wiki. ppl _↓	ARC _e acc	ARC _c acc _n	Hella. acc _n	Lamb. acc	PIQA acc	Wino. acc	Avg.
<i>1.3B parameters with 100B training tokens, L=24, d=2,048</i>										
Xfmr++	N/A	15.3	17.1	54.1	27.1	49.3	47.0	70.3	54.9	50.5
Mamba	$64 \times Ld$	15.4	17.3	57.1	28.2	50.3	44.4	71.8	52.3	50.7
RetNet	$512 \times Ld$	15.4	17.3	57.4	27.9	50.3	44.6	71.7	51.8	50.6
GLA	$256 \times Ld$	15.4	17.6	55.4	27.7	49.0	46.4	69.9	54.0	50.4
HGRN2	$128 \times Ld$	11.8	16.9	58.1	28.1	51.8	49.4	71.4	52.3	51.9
GSA	$128 \times Ld$	12.6	16.7	58.1	28.2	51.0	47.4	72.0	53.4	51.7
<i>2.7B parameters with 100B training tokens, L=32, d=2,560</i>										
Xfmr++	N/A	10.7	15.2	59.8	27.5	54.2	52.3	72.7	56.2	53.8
Mamba	$64 \times Ld$	13.6	15.9	60.7	29.8	53.9	46.4	72.8	53.9	52.9
RetNet	$512 \times Ld$	11.9	15.8	59.6	28.1	54.0	49.6	72.3	53.8	52.9
GLA	$256 \times Ld$	12.4	15.5	59.2	29.9	54.0	50.4	71.7	55.7	53.5
HGRN2	$128 \times Ld$	8.8	14.6	60.8	30.3	58.7	55.4	73.0	54.2	55.4
GSA	$128 \times Ld$	9.8	14.8	61.9	30.7	57.0	52.7	73.5	56.0	55.3

4.1.2 Results on in-context recall-intensive tasks

While subquadratic models can achieve comparable performance to (softmax-based) Transformers in language modeling and understanding tasks, their performance on recall-intensive tasks significantly lags behind Transformers and varies greatly across different subquadratic models, as observed in many recent studies [3, 4, 96, 97]. Therefore, it is crucial to improve linear models on in-context recall-intensive tasks.



(a) Results on the synthetic MQAR task. We adopt the most challenging settings in [2], utilizing a sequence length of 512 and 64 key-value pairs. Xfmr++ with standard attention achieves near-perfect results in this settings and is thus omitted for brevity.

(b) Results on the recall-intensive tasks used in [4]. We truncate the input to a maximum of 2K tokens.

	State size	FDA	SWDE	SQuAD	NQ	TriviaQA	Drop	Avg.
<i>1.3B params / 100B tokens, L=24, d=2048</i>								
Xfmr++	N/A	46.0	29.2	41.0	24.8	58.8	21.3	36.9
Mamba	$64 \times Ld$	13.9	25.4	33.2	18.5	53.5	21.7	27.7
RetNet	$512 \times Ld$	21.2	27.2	34.0	15.5	52.7	20.0	28.4
GLA	$256 \times Ld$	26.7	30.6	34.8	21.5	56.0	19.1	31.4
HGRN2	$128 \times Ld$	9.9	23.1	32.0	16.4	55.2	19.1	25.9
GSA	$128 \times Ld$	23.6	29.8	36.0	23.2	57.0	20.9	31.8
<i>2.7B params / 100B tokens, L=32, d=2560</i>								
Xfmr++	N/A	62.3	30.9	44.3	29.3	61.8	21.4	41.7
Mamba	$64 \times Ld$	21.5	26.7	34.2	21.2	57.0	22.2	30.5
RetNet	$512 \times Ld$	24.1	26.1	36.4	20.4	57.3	21.8	31.0
GLA	$256 \times Ld$	30.3	35.5	36.8	23.3	58.2	21.8	34.3
HGRN2	$128 \times Ld$	15.0	29.9	35.1	17.0	59.8	20.0	29.5
GSA	$128 \times Ld$	39.1	33.5	39.0	26.9	60.8	19.9	36.5

MQAR. We first present the results on the multi-query associative recall (MQAR) task [2], a diagnostic synthetic task that requires models to retrieve multiple associative key-value pairs from the context. This task has been shown to strongly correlate with language modeling performance [2]. The results in Table 3a validate the effectiveness of GSA.

Real-world tasks. Next, we evaluate the zero-shot in-context learning performance on recall-intensive tasks, as used in Arora et al. [4].⁶ Specifically, we assess information retrieval on FDA [93] and SWDE [49], which are designed to evaluate retrieval from in-context passages scraped from HTML/PDFs. We also evaluate question answering on SQuAD [70], NQ [46], TriviaQA [40], and Drop [20], where models must ground their answers in in-context documents.

As shown in Table 3b, Xfmr++ achieves the best average performance, as expected. Meanwhile, GSA outperforms all other subquadratic baseline models by a notable margin without requiring a larger state size. We believe this advantage stems from GSA’s context-aware memory read-out mechanism (as discussed in §2.2) and its forgetting mechanism (i.e., the gating mechanism), enabling it to manipulate finite-sized memory more effectively.

4.1.3 Ablation

Table 2 presents the results of our ablation studies. Our findings indicate that: (i) the inclusion of the gating mechanism in GSA is crucial for improving language modeling perplexity; (ii) applying softmax non-linearities after the first recurrent pass is beneficial; and (iii) using 64 slots strikes an optimal balance between performance and efficiency.⁷

4.1.4 Efficiency

Fig. 4a illustrates the training throughput for four models on a single H800 GPU⁸. To optimize memory usage, we employ the technique of recomputing the recurrent hidden state during the backward pass, as done in FLA [95] and Mamba2 [16]. This approach results in reduced memory consumption (Fig. 4b) at the cost of slightly lower training throughputs (Fig. 4a).

Despite requiring two GLA passes, GSA maintains comparable training throughputs to GLA due to its reduced state size. Since inference is primarily memory-bound, inference speed highly correlates with state size. As a result, GSA, with its smaller state size compared to RetNet and GLA, achieves faster inference speeds, as shown in Figure 4c.

4.2 Finetuning Pretrained Transformers to RNNs

The concept of finetuning pretrained Transformers to linear Transformers for recurrent inference was first introduced in T2R [41]. This approach uses pretrained language model weights to initialize all parameters, leveraging the similarity between linear attention and softmax attention, and finetunes all parameters, significantly reducing the total training time compared to training from scratch. Kasai et al. [41] also introduced a *parametric* feature map, implemented as a learnable MLP layer followed by ReLU, applied after the query/key projections. SUPRA, a follow-up to T2R, found that the original T2R approach did not perform well in the era of LLMs, and highlighted the importance of

Table 2: Ablation study results for 340M models trained on 10B Slimpajama tokens.

	PPL (↓)
GSA w/ 64 slots	13.51
<i>Ablations on gating mechanism</i>	
w/o decay (i.e., ABC)	16.94
w/ data-independent decay	15.83
<i>Ablations on non-linearity</i>	
– softmax	14.03
– softmax + Swish	13.71
– softmax + ReLU	13.69
– softmax + ReLU ²	13.95
<i>Ablations on slot size</i>	
w/ 32 slots	13.74
w/ 128 slots	13.46

⁶Since our pretrained models are neither instruction-tuned nor instruction-aligned, following Arora et al. [4], we use their Cloze Completion Formatting prompts for evaluation. It is noteworthy that results for certain tasks may differ significantly from those obtained using lm-evaluation-harness [21] due to variations in prompt templates.

⁷Empirically, we found that 32, 64, and 128 slots result in training throughputs of 46.7K, 44.1K, and 37.1K tokens/s, respectively, under the settings described in the next section. Given the marginal improvement when increasing the slot size from 64 to 128, along with the significant slowdown in training, we chose 64 slots.

⁸We utilize the training throughput benchmark scripts provided by FLA [95] for our measurements.

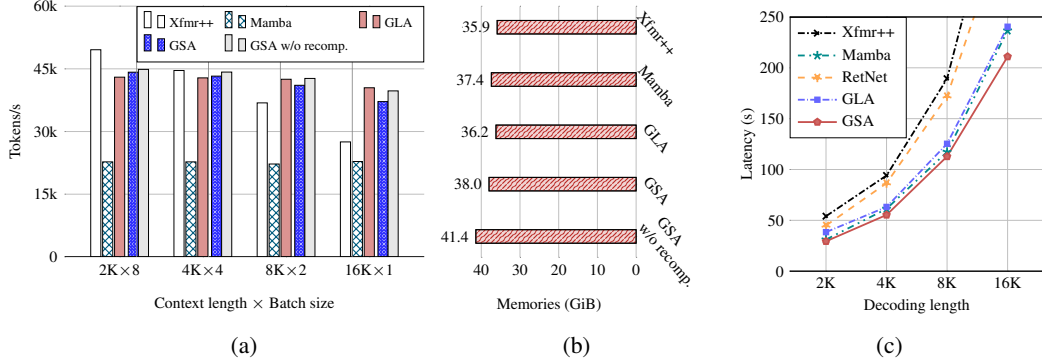


Figure 4: (a) Training throughput of various 1.3B models on a single H800 GPU, with a fixed batch size containing 16K tokens. “GSA w/o recomp.” indicates the use of the GSA kernel without hidden state recomputation during the backward pass. (b) Memory footprint (in GiB) of each 1.3B model during training with a batch size containing 16K tokens. (c) Inference latency (in seconds) of each 1.3B model on a single H800 GPU with 2K prefix tokens and a batch size of 1.

Table 3: Performance comparison across various 7B models. ♣ denotes models using softmax-attention. † denotes our results.

Shot(s)	Size	Tokens	ARC _e	ARC _c	Hella.	PIQA	Wino.	NQ	TriviaQA	BBH	MMLU	Avg.
			0	0	0	0	0	5	5	3	5	
<i>Models trained from scratch (for reference)</i>												
RWKV6	7B	1.4T	73.6	44.0	75.2	78.4	68.5	20.9	59.5	23.4	43.9	54.1
Mamba	7B	1.2T	77.6	46.8	77.8	81.0	72.3	25.4	66.2	21.5	33.2	55.7
Llama2♣	7B	2T	76.4	46.2	76.0	78.0	69.2	26.0	64.2	39.1	45.5	57.8
Gemma♣	7B	6T	81.5	53.2	80.5	79.8	74.0	24.3	63.7	58.9	63.2	64.3
Mistral♣	7B	?	80.8	54.0	81.1	80.6	74.0	29.7	70.3	56.5	62.4	65.5
<i>Models finetuned from Mistral 7B</i>												
SUPRA	7B	+20B	74.6	42.3	74.8	80.1	67.4	-	-	-	28.0	-
RetNet†	7B	+20B	73.3	39.9	72.9	77.8	66.1	16.2	43.0	8.7	26.1	47.1
GLA†	7B	+20B	74.6	44.0	75.9	79.2	69.5	22.2	57.8	20.8	28.4	52.5
GSA†	7B	+20B	75.9	43.9	76.5	78.7	70.1	23.4	60.7	23.5	32.4	53.9
SUPRA	7B	+100B	76.0	45.7	77.1	79.9	70.3	24.7	60.4	19.8	34.1	54.2
GSA†	7B	+100B	76.0	46.9	77.9	78.9	72.6	26.9	65.8	29.3	38.1	56.9

output normalization and a decay mechanism—adopted from RetNet [82]—as critical for finetuning performance. As a result, SUPRA essentially combines T2R and RetNet by finetuning pretrained Transformers into a RetNet architecture, though it excludes the Swish output gate.

Settings. In our preliminary experiments, we found that the learnable MLP layer was unnecessary and could be merged into the query and key projections, similar to the approach in Peng et al. [63]. We finetuned the pretrained Transformer Mistral 7B [39] to RetNet, as well as to GLA and GSA models. Following SUPRA, we add ReLU as the feature map activation for RetNet and GLA, which originally used an identity feature map without activation⁹, and also excluded the Swish output gate. For RetNet, there were no additional parameters; for GLA, the low-rank forget gate, and for GSA, the W_α matrix are trainable parameters, though both are small in parameter count and negligible in terms of the total model size. We set the peak learning rate to 3×10^{-5} with 1K steps of linear warmup following SUPRA. The training length was set to 2K tokens, with a batch size of 2M tokens. For convenience, we trained on the SlimPajama corpus, while SUPRA used RefineWeb [60], a higher-quality corpus. We leave the use of RefineWeb for future work.

⁹However, SUPRA reported poor performance with this strategy due to a significant discrepancy between training and finetuning, where an identity map can lead to negative attention scores, a pattern unseen in pretrained Transformers due to the nonnegativity of softmax.

Main results. Following Jiang et al. [39], Touvron et al. [87], we evaluated the models on common-sense reasoning tasks: ARC_e and ARC_c [14], Hellaswag [99], PIQA [8], and Winogrande [73]; world knowledge tasks: NQ [46] and TriviaQA [40]; and popular aggregated benchmarks: MMLU [31] and BBH [85]. Results are shown in Table 3. We observed a clear advantage in finetuning Mistral to GSA compared to GLA or RetNet, confirming our intuition that preserving softmax is beneficial in T2R settings. When trained with 100B tokens, Mistral-to-GSA outperforms RWKV6 and Mamba on average, even though those models were trained on over 1T tokens, thereby reducing the required training data size.

Long-context ability evaluation. Following Xiong et al. [94], we evaluated the models on long-sequence tasks, including Qasper [18], NarrativeQA [44], QuALITY [58], and QMSum [105]. For each task, the input was truncated to 16K tokens, which is $8\times$ the training length.

The results are shown in Table 4. Notably, GSA consistently outperforms other subquadratic models across all four tasks. We attribute this to the same factors observed in in-context recall-intensive task settings. Interestingly, Mistral-to-GSA also demonstrates overall better performance compared to RWKV6 and Mamba, which were trained from scratch on $>1T$ token.

Table 4: Long-context performance comparison.

	Qasper	NarrativeQA	QuALITY	QMSum
<i>Models trained from scratch (for reference)</i>				
RWKV6	9.2	14.4	30.8	1.1
Mamba	5.6	27.9	27.5	0.8
Mistral [✱]	25.8	25.1	38.0	5.0
<i>Models finetuned from Mistral 7B on 20B tokens</i>				
RetNet	11.1	0.0	26.2	0.0
GLA	18.4	17.2	30.9	9.0
GSA	18.8	19.2	32.0	10.0

5 Related works

Matrix-valued linear RNNs with hardware-efficient training. Traditional RNNs (e.g., LSTM [32], GRU [12]) maintain 1-dimensional hidden states, which are often too small to capture sufficient information. Recent work emphasizes the importance of expanding the size of recurrent states [29, 69, 96, 82, 61, 16]. However, naive state expansion dramatically increases FLOPs and I/O costs, making training impractical. To address this, Mamba introduces an I/O-aware approach, reducing I/O costs by materializing parameters and hidden states only on SRAM (instead of HBM). However, Mamba’s recurrence cannot be expressed in matmul form, leading to two key issues: (i) high FLOP count cannot be optimized via tensor cores (the GPU’s fast matmul unit), resulting in slower runtimes; and (ii) the recurrent hidden states cannot be compactly represented and must be materialized on SRAM during backpropagation, limiting the recurrent state size due to SRAM constraints.

Mamba2 [16] addresses these limitations by adopting a linear attention [42]-like approach that enables hardware-efficient training. Linear attention expands the state using outer products, allowing for both parallel attention-style computation and recurrent inference (also known as state-space duality in Mamba2). The chunkwise algorithm interpolates between parallel and recurrent forms, enabling hardware-efficient, linear-time training [33, 82, 96]. However, vanilla linear attention underperforms softmax attention in various tasks. Recent research has explored incorporating various decay or gating mechanisms to enhance model expressiveness and performance while maintaining matmul-based parallelism and chunkwise training. These include head-wise data-independent decay [82, 68]; head-wise data-dependent decay [62, 16, 6, 83]; and channel-wise data-dependent decay [96, 52, 43, 69, 61]. GSA leverages two-pass gated linear attention to further enhance capacity while allowing hardware-efficient training.

Fast weight RNNs. Fast weight programming [77], a classical concept intensively investigated in deep learning [5, 103, 74, 76, 56, 75, 35, 36, 52], has been shown to be closely related to (linear) Transformers [75]. The core idea involves using a slow network to produce rapid context-dependent weight modifications for the fast network. In linear attention, the fast network is a single-layer FFN with weight matrix S_t (Eq. 5), while the slow networks are the query/key/value projections.

Linear attention is known to suffer from limited memory capacity [75], potentially due to the constraints of a single-layer FFN without a large representation. In contrast, ABC and GSA can be viewed as implementing a two-layer fast FFN with either additive update rule or gated update rule [74, 52], where the weight matrices are $\tilde{K}_t$ and $\tilde{V}_t$ connected by the softmax activation function (Eq. 3 and Eq. 6). This structure resembles DeltaMLP [35], which uses a delta update rule [92, 75, 98]

and a multi-layer (potentially beyond two layers) fast FFN. The greater capacity of a two-layer FFN compared to a similarly sized single-layer FFN could explain why GSA requires a smaller state size to achieve similar or even better performance, especially in long sequence and recall-intensive tasks.

Finetuning Transformers to RNNs. As discussed, this paradigm could significantly reduce the training cost for large-scale recurrent language models. The idea of distilling Transformers to RNNs to improve inference efficiency can be traced back to Gerstenberger et al. [23]. In the following, we briefly introduce some recent works that complement those already mentioned in §4.2. Zhang et al. [101] highlight the desirable properties of softmax, such as attention spikiness and dot-product monotonicity, and employ a learnable MLP layer to approximate softmax behavior using logit distillation loss (while freezing other parameters). Chen et al. [10] introduce DiJiang, an effective method for approximating attention distributions using the Discrete Cosine Transform (DCT) to enable frequency-domain kernelization, leading to faster feature mapping. Bick et al. [7] propose a multi-stage distillation approach, aligning attention distributions (similar to Hedgehog [101]), hidden states, and output logits to transfer knowledge from a pretrained Transformer teacher to a student Mamba model. Wang et al. [90] distill Transformer-based LLMs into hybrid Mamba-Attention architectures in the spirit of Ren et al. [71], Lieber et al. [47], Waleffe et al. [89]. However, they freeze the FFN weights, while Choi [13] suggest that it might be more effective to unfreeze them. In this work, we highlight the importance of the softmax operator, as discussed in Zhang et al. [101], except that GSA directly incorporates softmax, while Zhang et al. [101] learns a feature map to mimic softmax, without actually including any softmax operator in the resulting model.

6 Limitations and future work

Due to the relatively small scale of our pretrained models (compared to large-scale models trained on trillions of tokens), we did not report any results on long-context tasks, as the performance would all be poor. However, we believe Table 4 provides positive indications of GSA’s long-context capabilities, and training on a larger token horizon and with larger models would address this. For copy-oriented tasks, we observed negative results on the Phonebook Lookup [38] and Needle-In-Haystack evaluations compared to Transformers, revealing the fundamental limitations of linear recurrent models in handling “precise local token shifts and comparison”, as discussed in Arora et al. [3]. Nonetheless, we expect this limitation could be significantly mitigated by pretraining a **hybrid GSA-attention model**, as recently explored [3, 71, 89, 47, 98], or by distilling pretrained Transformers into hybrid GSA-attention models, as in Wang et al. [90], or using different training objectives with JRT prompts, as in Arora et al. [4], or combining with YOCO [83, 25].

GSA follows GLA in using a gated update rule, although we acknowledge recent work on Parallel DeltaNet [98], which parallelizes the delta update rule computations in DeltaNet [75] over sequence length, significantly enhancing training efficiency. The delta rule is known to improve in-context retrieval ability [75, 98], aligning with one of the objectives of this work. We did not explore the analogous two-pass DeltaNet, but we leave this for future investigation, which would bring the approach closer to the original DeltaMLP [35], as discussed earlier. It would also be beneficial to compare GSA with more recent strong RNN models, such as **xLSTM** [6], **Mamba2** [16], **TTT** [84], and **Longhorn** [48].

7 Conclusions

This work introduces Gated Slot Attention (GSA), which enhances ABC [63] with a gating mechanism inspired by Gated Linear Attention (GLA [96]). By framing GSA as a two-pass GLA, we can leverage hardware-efficient implementations of GLA [95] to train GSA. As such, GSA benefits from context-aware memory reading and forgetting, implicitly increasing the model’s capacity despite a small actual state size, which improves training and inference efficiency. Through extensive experiments, we demonstrate the advantages of GSA in in-context recall-intensive tasks [4] and in “finetuning pretrained Transformers to RNNs” [41] scenarios.

Acknowledgments

We would like to thank Zhen Qin and Yikang Shen for their insightful discussions, Houquan Zhou and Kazuki Irie for providing valuable feedback on this manuscript.

We gratefully acknowledge the support by LuxiTech for computational resources; and the support by the National Natural Science Foundation of China (No. 62076173, 62476187), the High-level Entrepreneurship and Innovation Plan of Jiangsu Province (No. JSSCRC2021524), and the Project Funded by the Priority Academic Program Development of Jiangsu Higher Education Institutions. Yu Zhang was partially supported by Tencent AI Lab under Wei Bi’s mentorship. Songlin Yang was supported by Xianhong Wu Fellowship from MIT.

References

- [1] E. Akyürek, B. Wang, Y. Kim, and J. Andreas. In-context language learning: Architectures and algorithms. In *Proceedings of ICML*, 2024. URL <https://openreview.net/forum?id=3Z9CRr5srL>.
- [2] S. Arora, S. Eyuboglu, A. Timalsina, I. Johnson, M. Poli, J. Zou, A. Rudra, and C. Ré. Zoology: Measuring and improving recall in efficient language models, 2023.
- [3] S. Arora, S. Eyuboglu, M. Zhang, A. Timalsina, S. Alberti, D. Zinsley, J. Zou, A. Rudra, and C. Ré. Simple linear attention language models balance the recall-throughput tradeoff, 2024.
- [4] S. Arora, A. Timalsina, A. Singhal, B. Spector, S. Eyuboglu, X. Zhao, A. Rao, A. Rudra, and C. Ré. Just read twice: closing the recall gap for recurrent language models, 2024. URL <https://arxiv.org/abs/2407.05483>.
- [5] J. Ba, G. Hinton, V. Mnih, J. Z. Leibo, and C. Ionescu. Using fast weights to attend to the recent past, 2016.
- [6] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter. xlstm: Extended long short-term memory, 2024.
- [7] A. Bick, K. Y. Li, E. P. Xing, J. Z. Kolter, and A. Gu. Transformers to ssms: Distilling quadratic knowledge to subquadratic models. 2024. URL <https://api.semanticscholar.org/CorpusID:271903923>.
- [8] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi. Piqa: Reasoning about physical commonsense in natural language. In *In Proceedings of AAAI*, 2020.
- [9] G. E. Blelloch. Prefix sums and their applications. Technical report, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, 1993. URL <https://www.cs.cmu.edu/~guyb/papers/Ble93.pdf>.
- [10] H. Chen, Z. Liu, X. Wang, Y. Tian, and Y. Wang. Dijiang: Efficient large language models through compact kernelization. *ArXiv*, abs/2403.19928, 2024. URL <https://api.semanticscholar.org/CorpusID:268793982>.
- [11] T. Chen, B. Xu, C. Zhang, and C. Guestrin. Training deep nets with sublinear memory cost, 2016.
- [12] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of EMNLP*, pages 1724–1734, 2014. URL <https://aclanthology.org/D14-1179>.
- [13] S. Choi. Cross-architecture transfer learning for linear-cost inference transformers, 2024. URL <https://arxiv.org/abs/2404.02684>.
- [14] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.

- [15] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- [16] T. Dao and A. Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *CoRR*, abs/2405.21060, 2024. doi: 10.48550/ARXIV.2405.21060. URL <https://doi.org/10.48550/arXiv.2405.21060>.
- [17] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in NIPS*, pages 16344–16359, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf.
- [18] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, pages 4599–4610, 2021. URL <https://aclanthology.org/2021.naacl-main.365>.
- [19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier. Language modeling with gated convolutional networks. In *Proceedings of ICML*, pages 933–941, 2017. URL <https://proceedings.mlr.press/v70/dauphin17a.html>.
- [20] D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of NAACL*, pages 2368–2378, 2019. URL <https://aclanthology.org/N19-1246>.
- [21] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac’h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou. A framework for few-shot language model evaluation, 2023. URL <https://zenodo.org/records/10256836>.
- [22] F. Gers, J. Schmidhuber, and F. Cummins. Learning to forget: continual prediction with lstm. In *Proceedings of ICANN*, pages 850–855, 1999.
- [23] A. Gerstenberger, K. Irie, P. Golik, E. Beck, and H. Ney. Domain robust, fast, and compact neural language models. In *Proceedings of ICASSP*, Barcelona, Spain, 2020.
- [24] M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of EMNLP*, pages 5484–5495, Online and Punta Cana, Dominican Republic, 2021. URL <https://aclanthology.org/2021.emnlp-main.446>.
- [25] D. Goldstein, F. Obeid, E. Alcaide, G. Song, and E. Cheah. Goldfinch: High performance rwkv/transformer hybrid with linear pre-fill and extreme kv-cache compression. *ArXiv*, abs/2407.12077, 2024. URL <https://api.semanticscholar.org/CorpusID:271244694>.
- [26] A. Graves. Generating sequences with recurrent neural networks, 2014.
- [27] A. Graves, G. Wayne, and I. Danihelka. Neural turing machines, 2014.
- [28] R. Grazi, J. N. Siems, S. Schrodi, T. Brox, and F. Hutter. Is mamba capable of in-context learning? *ArXiv*, abs/2402.03170, 2024. URL <https://api.semanticscholar.org/CorpusID:267412719>.
- [29] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023.
- [30] A. Gu, C. Gulcehre, T. Paine, M. Hoffman, and R. Pascanu. Improving the gating mechanism of recurrent neural networks. In H. D. III and A. Singh, editors, *Proceedings of ICML*, pages 3800–3809. PMLR, 2020. URL <https://proceedings.mlr.press/v119/gu20a.html>.
- [31] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding, 2021. URL <https://arxiv.org/abs/2009.03300>.
- [32] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.

- [33] W. Hua, Z. Dai, H. Liu, and Q. Le. Transformer quality in linear time. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, editors, *Proceedings of ICML*, pages 9099–9117. PMLR, 2022. URL <https://proceedings.mlr.press/v162/hua22a.html>.
- [34] K. Irie, A. Gerstenberger, R. Schlüter, and H. Ney. How much self-attention do we need? Trading attention for feed-forward layers. In *Proceedings of ICASSP*, Virtual only, May 2020.
- [35] K. Irie, I. Schlag, R. Csord’as, and J. Schmidhuber. Going beyond linear transformers with recurrent fast weight programmers. *ArXiv*, abs/2106.06295, 2021. URL <https://api.semanticscholar.org/CorpusID:235417174>.
- [36] K. Irie, I. Schlag, R. Csord’as, and J. Schmidhuber. A modern self-referential weight matrix that learns to modify itself. In *International Conference on Machine Learning*, 2022. URL <https://api.semanticscholar.org/CorpusID:246823084>.
- [37] S. Jelassi, D. Brandfonbrener, S. M. Kakade, and E. Malach. Repeat after me: Transformers are better than state space models at copying. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=duRRoGeoQT>.
- [38] S. Jelassi, D. Brandfonbrener, S. M. Kakade, and E. Malach. Repeat after me: Transformers are better than state space models at copying, 2024.
- [39] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. Mistral 7b, 2023.
- [40] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, *Proceedings of ACL*, pages 1601–1611, Vancouver, Canada, July 2017. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147>.
- [41] J. Kasai, H. Peng, Y. Zhang, D. Yogatama, G. Ilharco, N. Pappas, Y. Mao, W. Chen, and N. A. Smith. Finetuning pretrained transformers into RNNs. In M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, editors, *Proceedings of EMNLP*, pages 10630–10643, 2021. URL <https://aclanthology.org/2021.emnlp-main.830>.
- [42] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In H. D. III and A. Singh, editors, *Proceedings of ICML*, pages 5156–5165. PMLR, 2020. URL <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- [43] T. Katsch. Gateloop: Fully data-controlled linear recurrence for sequence modeling, 2024.
- [44] T. Kočiský, J. Schwarz, P. Blunsom, C. Dyer, K. M. Hermann, G. Melis, and E. Grefenstette. The NarrativeQA reading comprehension challenge. *TACL*, pages 317–328, 2018. URL <https://aclanthology.org/Q18-1023>.
- [45] D. Krotov and J. Hopfield. Large Associative Memory Problem in Neurobiology and Machine Learning, 2021. URL <http://arxiv.org/abs/2008.06996>.
- [46] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov. Natural questions: A benchmark for question answering research. *TACL*, pages 452–466, 2019. URL <https://aclanthology.org/Q19-1026>.
- [47] O. Lieber, B. Lenz, H. Bata, G. Cohen, J. Osin, I. Dalmedigos, E. Safahi, S. Meirom, Y. Belinkov, S. Shalev-Shwartz, O. Abend, R. Alon, T. Asida, A. Bergman, R. Glozman, M. Gokhman, A. Manevich, N. Ratner, N. Rozen, E. Shwartz, M. Zusman, and Y. Shoham. Jamba: A hybrid transformer-mamba language model, 2024. URL <https://arxiv.org/abs/2403.19887>.

- [48] B. Liu, R. Wang, L. Wu, Y. Feng, P. Stone, and Q. Liu. Longhorn: State space models are amortized online learners. *ArXiv*, abs/2407.14207, 2024. URL <https://api.semanticscholar.org/CorpusID:271310065>.
- [49] C. Lockard, P. Shiralkar, and X. L. Dong. OpenCeres: When Open Information Extraction Meets the Semi-Structured Web. In J. Burstein, C. Doran, and T. Solorio, editors, *Proceedings of NAACL*, pages 3047–3056, Minneapolis, Minnesota, 2019. doi: 10.18653/v1/N19-1309. URL <https://aclanthology.org/N19-1309>.
- [50] I. Loshchilov and F. Hutter. Decoupled weight decay regularization, 2019.
- [51] X. Ma, X. Kong, S. Wang, C. Zhou, J. May, H. Ma, and L. Zettlemoyer. Luna: Linear unified nested attention. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, *Advances in NIPS*, volume 34, pages 2441–2453. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/14319d9cfc6123106878dc20b94fbaf3-Paper.pdf.
- [52] H. H. Mao. Fine-tuning pre-trained transformers into decaying fast weights. In Y. Goldberg, Z. Kozareva, and Y. Zhang, editors, *Proceedings of EMNLP*, pages 10236–10242, Abu Dhabi, United Arab Emirates, 2022. URL <https://aclanthology.org/2022.emnlp-main.697>.
- [53] E. Martin and C. Cundy. Parallelizing linear recurrent neural nets over sequence length. In *Proceedings of ICLR*, 2018. URL <https://openreview.net/forum?id=HyUNwulC->.
- [54] J. Mercat, I. Vasiljevic, S. Keh, K. Arora, A. Dave, A. Gaidon, and T. Kollar. Linearizing large language models, 2024.
- [55] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models, 2016.
- [56] T. Munkhdalai, A. Sordoni, T. Wang, and A. Trischler. Metalearned neural memory. *ArXiv*, abs/1907.09720, 2019. URL <https://api.semanticscholar.org/CorpusID:198179407>.
- [57] M. Oren, M. Hassid, Y. Adi, and R. Schwartz. Transformers are multi-state rnns, 2024.
- [58] R. Y. Pang, A. Parrish, N. Joshi, N. Nangia, J. Phang, A. Chen, V. Padmakumar, J. Ma, J. Thompson, H. He, and S. R. Bowman. Quality: Question answering with long input texts, yes!, 2022. URL <https://arxiv.org/abs/2112.08608>.
- [59] D. Paperno, G. Kruszewski, A. Lazaridou, Q. N. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, and R. Fernández. The lambda dataset, 2016.
- [60] G. Penedo, Q. Malartic, D. Hesslow, R. Cojocaru, A. Cappelli, H. Alobeidli, B. Pannier, E. Almazrouei, and J. Launay. The RefinedWeb dataset for Falcon LLM: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*, 2023. URL <https://arxiv.org/abs/2306.01116>.
- [61] B. Peng, D. Goldstein, Q. Anthony, A. Albalak, E. Alcaide, S. Biderman, E. Cheah, X. Du, T. Ferdinan, H. Hou, P. Kazienko, K. K. GV, J. Kocoń, B. Koptyra, S. Krishna, R. M. J. au2, N. Muennighoff, F. Obeid, A. Saito, G. Song, H. Tu, S. Woźniak, R. Zhang, B. Zhao, Q. Zhao, P. Zhou, J. Zhu, and R.-J. Zhu. Eagle and finch: RwkV with matrix-valued states and dynamic recurrence, 2024.
- [62] H. Peng, N. Pappas, D. Yogatama, R. Schwartz, N. Smith, and L. Kong. Random feature attention. In *Proceedings of ICLR*, 2021. URL <https://openreview.net/forum?id=QtTKTdVrFBB>.
- [63] H. Peng, J. Kasai, N. Pappas, D. Yogatama, Z. Wu, L. Kong, R. Schwartz, and N. A. Smith. ABC: Attention with bounded-memory control. In *Proceedings of ACL*, pages 7469–7483, 2022. URL <https://aclanthology.org/2022.acl-long.515>.
- [64] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference, 2022.
- [65] S. Pramanik, E. Elelimy, M. C. Machado, and A. White. Recurrent linear transformers, 2023. URL <https://arxiv.org/abs/2310.15719>.

- [66] Z. Qin, X. Han, W. Sun, D. Li, L. Kong, N. Barnes, and Y. Zhong. The devil in linear transformer. In Y. Goldberg, Z. Kozareva, and Y. Zhang, editors, *Proceedings of EMNLP*, pages 7025–7041, Abu Dhabi, United Arab Emirates, 2022. doi: 10.18653/v1/2022.emnlp-main.473. URL <https://aclanthology.org/2022.emnlp-main.473>.
- [67] Z. Qin, S. Yang, and Y. Zhong. Hierarchically gated recurrent neural network for sequence modeling. In *Advances in NIPS*, 2023. URL <https://openreview.net/forum?id=P1TCHxJwLB>.
- [68] Z. Qin, D. Li, W. Sun, W. Sun, X. Shen, X. Han, Y. Wei, B. Lv, X. Luo, Y. Qiao, and Y. Zhong. Transormerllm: A faster and better large language model with improved transormer, 2024.
- [69] Z. Qin, S. Yang, W. Sun, X. Shen, D. Li, W. Sun, and Y. Zhong. Hgrn2: Gated linear rnns with state expansion, 2024.
- [70] P. Rajpurkar, R. Jia, and P. Liang. Know What You Don’t Know: Unanswerable Questions for SQuAD. In *Proceedings of ACL*, Melbourne, Australia, 2018. Association for Computational Linguistics.
- [71] L. Ren, Y. Liu, Y. Lu, Y. Shen, C. Liang, and W. Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *CoRR*, abs/2406.07522, 2024. doi: 10.48550/ARXIV.2406.07522. URL <https://doi.org/10.48550/arXiv.2406.07522>.
- [72] A. Rush. Torch-struct: Deep structured prediction library. In A. Celikyilmaz and T.-H. Wen, editors, *Proceedings of ACL*, pages 335–342, Online, July 2020. doi: 10.18653/v1/2020.acl-demos.38. URL <https://aclanthology.org/2020.acl-demos.38>.
- [73] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019. URL <https://arxiv.org/abs/1907.10641>.
- [74] I. Schlag and J. Schmidhuber. Gated fast weights for on-the-fly neural program generation. In *Proceedings of ICLR*, 2017. URL <https://api.semanticscholar.org/CorpusID:216094255>.
- [75] I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers. In M. Meila and T. Zhang, editors, *Proceedings of ICML*, pages 9355–9366. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/schlag21a.html>.
- [76] I. Schlag, T. Munkhdalai, and J. Schmidhuber. Learning associative inference using fast weight memory, 2021. URL <https://arxiv.org/abs/2011.07831>.
- [77] J. Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- [78] N. Shazeer. Glu variants improve transformer, 2020.
- [79] D. Soboleva, F. Al-Khateeb, R. Myers, J. R. Steeves, J. Hestness, and N. Dey. Slimpajama: A 627b token cleaned and deduplicated version of redpajama, 2023. URL <https://huggingface.co/datasets/cerebras/SlimPajama-627B>.
- [80] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [81] S. Sukhbaatar, a. szlam, J. Weston, and R. Fergus. End-to-end memory networks. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in NIPS*. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/file/8fb21ee7a2207526da55a679f0332de2-Paper.pdf.
- [82] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei. Retentive network: A successor to transformer for large language models, 2023.
- [83] Y. Sun, L. Dong, Y. Zhu, S. Huang, W. Wang, S. Ma, Q. Zhang, J. Wang, and F. Wei. You only cache once: Decoder-decoder architectures for language models, 2024. URL <https://arxiv.org/abs/2405.05254>.

- [84] Y. Sun, X. Li, K. Dalal, J. Xu, A. Vikram, G. Zhang, Y. Dubois, X. Chen, X. Wang, O. Koyejo, T. Hashimoto, and C. Guestrin. Learning to (learn at test time): Rnns with expressive hidden states. *ArXiv*, abs/2407.04620, 2024. URL <https://api.semanticscholar.org/CorpusID:271039606>.
- [85] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. Le, E. Chi, D. Zhou, and J. Wei. Challenging BIG-bench tasks and whether chain-of-thought can solve them. In *Findings of the ACL*, pages 13003–13051, Toronto, Canada, 2023. URL <https://aclanthology.org/2023.findings-acl.824>.
- [86] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. Llama: Open and efficient foundation language models, 2023.
- [87] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023. URL <https://arxiv.org/abs/2307.09288>.
- [88] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in NIPS*. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [89] R. Waleffe, W. Byeon, D. Riach, B. Norick, V. Korthikanti, T. Dao, A. Gu, A. Hatamizadeh, S. Singh, D. Narayanan, G. Kulshreshtha, V. Singh, J. Casper, J. Kautz, M. Shoenybi, and B. Catanzaro. An empirical study of mamba-based language models, 2024. URL <https://arxiv.org/abs/2406.07887>.
- [90] J. Wang, D. Paliotta, A. May, A. M. Rush, and T. Dao. The mamba in the llama: Distilling and accelerating hybrid models, 2024. URL <https://arxiv.org/abs/2408.15237>.
- [91] K. Wen, X. Dang, and K. Lyu. Rnns are not transformers (yet): The key bottleneck on in-context retrieval. *ArXiv*, abs/2402.18510, 2024. URL <https://api.semanticscholar.org/CorpusID:268041425>.
- [92] B. Widrow and M. E. Hoff. Adaptive switching circuits. 1988. URL <https://api.semanticscholar.org/CorpusID:60830585>.
- [93] E. Wu, K. Wu, R. Daneshjou, D. Ouyang, D. E. Ho, and J. Zou. How medical AI devices are evaluated: limitations and recommendations from an analysis of FDA approvals. *Nature Medicine*, pages 582–584, 2021. URL <https://doi.org/10.1038/s41591-021-01312-x>.
- [94] W. Xiong, J. Liu, I. Molybog, H. Zhang, P. Bhargava, R. Hou, L. Martin, R. Rungta, K. A. Sankararaman, B. Oguz, M. Khabsa, H. Fang, Y. Mehdad, S. Narang, K. Malik, A. Fan, S. Bhosale, S. Edunov, M. Lewis, S. Wang, and H. Ma. Effective long-context scaling of foundation models, 2023. URL <https://arxiv.org/abs/2309.16039>.
- [95] S. Yang and Y. Zhang. FLA: A Triton-Based Library for Hardware-Efficient Implementations of Linear Attention Mechanism, 2024. URL <https://github.com/sustcsonglin/flash-linear-attention>.
- [96] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim. Gated linear attention transformers with hardware-efficient training. In *Proceedings of ICML*. PMLR, 2024.

- [97] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. Parallelizing linear transformers with the delta rule over sequence length. *CoRR*, abs/2406.06484, 2024. doi: 10.48550/ARXIV.2406.06484. URL <https://doi.org/10.48550/arXiv.2406.06484>.
- [98] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. Parallelizing linear transformers with the delta rule over sequence length. *ArXiv*, abs/2406.06484, 2024. URL <https://api.semanticscholar.org/CorpusID:270371554>.
- [99] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [100] J. Zhang, S. Jiang, J. Feng, L. Zheng, and L. Kong. Cab: Comprehensive attention benchmarking on long sequence modeling. *ArXiv*, abs/2210.07661, 2022. URL <https://api.semanticscholar.org/CorpusID:252907545>.
- [101] M. Zhang, K. Bhatia, H. Kumbong, and C. Ré. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry, 2024.
- [102] P. Zhang, G. Zeng, T. Wang, and W. Lu. Tinyllama: An open-source small language model, 2024.
- [103] W. Zhang and B. Zhou. Learning to update auto-associative memory in recurrent neural networks for improving sequence memorization. *ArXiv*, abs/1709.06493, 2017. URL <https://api.semanticscholar.org/CorpusID:22458497>.
- [104] Y. Zhang and D. Cai. Linearizing transformer with key-value memory. In Y. Goldberg, Z. Kozareva, and Y. Zhang, editors, *Proceedings of EMNLP*, pages 346–359, Abu Dhabi, United Arab Emirates, 2022. doi: 10.18653/v1/2022.emnlp-main.24. URL <https://aclanthology.org/2022.emnlp-main.24>.
- [105] M. Zhong, D. Yin, T. Yu, A. Zaidi, M. Mutuma, R. Jha, A. H. Awadallah, A. Celikyilmaz, Y. Liu, X. Qiu, and D. Radev. QMSum: A new benchmark for query-based multi-domain meeting summarization. In *Proceedings of NAACL*, pages 5905–5921, Online, 2021. doi: 10.18653/v1/2021.naacl-main.472. URL <https://aclanthology.org/2021.naacl-main.472>.
- [106] G.-B. Zhou, J. Wu, C.-L. Zhang, and Z.-H. Zhou. Minimal gated unit for recurrent neural networks, 2016.

A Linear Attention and its Chunkwise Form

Linear Attention (LA) [42, 66, 68] emerges as an alternative to resolve the quadratic complexity of self-attention (SA). The key idea is to use the *kernel trick*, which replaces softmax with a decomposable kernel function, resulting the following *parallel form*:¹⁰

$$\mathbf{O} = ((\phi(\mathbf{Q})\phi(\mathbf{K})^\top) \odot \mathbf{M})\mathbf{V}. \quad (10)$$

where $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ functions as feature mapping applied to each input. Unfolding Eq. 10, we have

$$\begin{aligned} \mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t &= \mathbf{W}_q \mathbf{x}_t, \mathbf{W}_k \mathbf{x}_t, \mathbf{W}_v \mathbf{x}_t \in \mathbb{R}^d, \\ \mathbf{o}_t &= \sum_{i=1}^t \mathbf{v}_i f(\mathbf{k}_i^\top \mathbf{q}_t) = \sum_{i=1}^t \mathbf{v}_i \phi(\mathbf{k}_i)^\top \phi(\mathbf{q}_t) = \left[\mathbf{S}_t \equiv \sum_{i=1}^t \phi(\mathbf{k}_i) \otimes \mathbf{v}_i \right]^\top \phi(\mathbf{q}_t). \end{aligned} \quad (11)$$

$\otimes$ means outer product operation. It is clear that by leveraging the associativity, LA admits simple recurrent updating rules with matrix-valued hidden states $\mathbf{S}_t \in \mathbb{R}^{m \times d}$:

$$\mathbf{o}_t = \mathbf{S}_t^\top \phi(\mathbf{q}_t); \mathbf{S}_t = \mathbf{S}_{t-1} + \phi(\mathbf{k}_t) \otimes \mathbf{v}_t. \quad (12)$$

By reserving bounded m memory slots only, the overall computation complexity is reduced from $O(T^2 d)$ to $O(Tmd)$. When the sequence length is $T \gg m, d$, the md factor has a minor impact on the complexity, and LA can be much more efficient than its counterpart with quadratic complexity.

During inference, LA enjoys the merits of RNNs, which only need to maintain $O(md)$ hidden memories, helping avoid the memory-cost KV cache management in SA mechanisms. However, Eq. 12 employs a simple additive updating rule and can be hard to “forget” unrelated information if necessary [62], making the limited memory states vulnerable to be chaotic.

Gating mechanism has played a key role in classical RNNs [32, 22, 12], which serves as a mechanism to control the information flows in the network and help read and write from the memory selectively. [82] propose to apply a *data-independent* gate to LA, significantly narrowing the gap between LA and SA: $\mathbf{S}_t = \lambda \mathbf{S}_{t-1} + \phi(\mathbf{k}_t) \otimes \mathbf{v}_t$, $\lambda \in [0, 1]$ is a non-learnable scalar. Recent work [96, 43] further imposes a finer-grained *data-dependent* gate:

$$\mathbf{S}_t = \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1} + \phi(\mathbf{k}_t) \otimes \mathbf{v}_t, \quad (13)$$

where each $\alpha_t \in [0, 1]^m$ from $\mathbf{A} := \{\boldsymbol{\alpha}_i\}_{i=1}^T \in [0, 1]^{T \times m}$ is dependent on the input. Alternatively, we can couple the key values with the forget gates by allowing $\phi(\mathbf{k}_t) = 1 - \boldsymbol{\alpha}_t$ in spirit of [12, 106] and [69], which reduces the number of parameters and improves efficiency accordingly.

A.1 Hardware-Efficient Training

Despite the theoretical advantages of linear complexity, the recurrent form of Eq. 12 is still inefficient during training. Such recurrent computation prevents the full utilization of modern GPU parallelism over sequence lengths [53, 72]. On the other hand, the parallel form (Eq. 10) can be parallelized in similar vein as in flash attention [17, 15]. However, due to the existence of the casual mask $\mathbf{M}$, we can not rearrange its computation order by KV first, so that the parallel form still adheres to the quadratic complexity, which can hardly be scaled to very-long training context (e.g., sequences with more than 8K tokens).

Chunkwise form recurrences have been carried forward by [82], and achieve a good trade-off between the recurrent and parallel forms. [96] further disclose that the element-wise gating of Eq. 13 also satisfies the associative property required by parallel scan [9] and derive a parallelized chunkwise gated linear attention in a similar vein. The key idea is to partition the sequence into $N = \lceil \frac{T}{C} \rceil$ chunks of size C with $\mathbf{Q}_{[t]} = \mathbf{q}_{tC}, \mathbf{q}_{tC+1}, \dots, \mathbf{q}_{tC+C}$, and so forth for $\mathbf{K}_{[t]}, \mathbf{V}_{[t]} \in \mathbb{R}^{C \times d}, \mathbf{A}_{[t]} \in \mathbb{R}^{C \times m}$. Firstly, unrolling the i -th hidden state in the t -th chunk in Eq. 13, we get

$$\begin{aligned} \mathbf{S}_{[t],i} &= \text{Diag}(\mathbf{A}_{[t],i}) \mathbf{S}_{[t],i-1} + \phi(\mathbf{K}_{[t],i}) \otimes \mathbf{V}_{[t],i} = \dots \\ &= \text{Diag} \left(\prod_{j=1}^i \mathbf{A}_{[t],j} \right) \mathbf{S}_{[t-1],C} + \sum_{k=1}^i \left(\phi(\mathbf{K}_{[t],k}) \odot \prod_{j=k+1}^i \mathbf{A}_{[t],j} \right) \otimes \mathbf{V}_{[t],k} \end{aligned} \quad (14)$$

¹⁰There is a normalization term in vanilla LA similar to softmax, [66] reveal that removing it could avoid potential gradient explosions.

We write the last hidden in the chunk $\mathbf{S}_{[t],C}$ as $\mathbf{S}_{[t]}$ interchangeably for simplicity. Define $\vec{\mathcal{A}}_{[t],i} = \prod_{j=1}^i \mathbf{A}_{[t],j} \in [0, 1]^d$ as the cumulative decay from the start of chunk to i , and likewise $\overleftarrow{\mathcal{A}}_{[t],i} = \prod_{j=i+1}^C \mathbf{A}_{[t],j} \in [0, 1]^d$ from $i + 1$ to the end of the chunk, then

$$\mathbf{S}_{[t]} = \text{Diag}(\vec{\mathcal{A}}_{[t],C}) \mathbf{S}_{[t-1]} + (\mathbf{K}_{[t]} \odot \overleftarrow{\mathcal{A}}_{[t]})^\top \mathbf{V}_{[t]} \quad (15)$$

$\vec{\mathcal{A}}, \overleftarrow{\mathcal{A}}$ can be absorbed into $\mathbf{Q}, \mathbf{K}$ first: $\bar{\mathbf{Q}}_{[t]} = \phi(\mathbf{Q}_{[t]}) \odot \vec{\mathcal{A}}_{[t]}$, $\bar{\mathbf{K}}_{[t]} = \phi(\mathbf{K}_{[t]}) \odot (\overleftarrow{\mathcal{A}}_{[t]} / \vec{\mathcal{A}}_{[t],C})$. Combining them with Eq. 10 and Eq. 14, we derive the following vectorized updating rules

$$\mathbf{O}_{[t]} = \bar{\mathbf{Q}}_{[t]} \mathbf{S}_{[t-1]} + (\bar{\mathbf{Q}}_{[t]} \bar{\mathbf{K}}_{[t]}^\top \odot \mathbf{M}_{[t]}) \mathbf{V}_{[t]} \quad (16)$$

The first term is referred to as the *inter* chunk part and the second term is the *intra* chunk part. The process to get this *intra* part is a little more involved as the cumulative productions of $\overleftarrow{\mathcal{A}}_{[t]} / \vec{\mathcal{A}}_{[t],C}$ is greater than 1, which can lead to numerical instability. [96] deal with this issue by proposing a secondary-chunking strategy, and we refer readers to their paper for more details.

Hardware considerations Modern GPU architectures, such as the NVIDIA A100, offer highly optimized matrix multiplication (matmul) operations through specialized Tensor Cores, achieving up to $16\times$ higher throughput than non-matmul operations [17]. However, this incurs IO overheads due to data transfer from slower, off-chip global high bandwidth memory (HBM) to on-chip shared memory (SRAM). The chunkwise form balances I/O and computation complexity tradeoffs. As shown in Eq.16, it improves parallelism over the sequence dimension while reducing non-matmul FLOPs greatly. Also, the chunk recurrent updating conducts the query and hidden states reduction in an online manner, requiring only $O(Ndm)$ hidden states materialized into HBMs, so that it can significantly reduce the memory/IO overheads. While LA enjoys much lower overall running FLOPs than SA, the chunkwise form displays a practical significant wall-clock speedup against SA, due to its hardware-efficient implementations [95].

B Details for GSA

Beyond the recurrent GSA form provided in Figure. 1, we give detailed, hardware-efficient procedures for the forward and backward passes of Gated Slot Attention (GSA) in Algorithm 1. For simplicity, we define $\mathbf{A} = \{\alpha_i\}_{i=1}^T \in [0, 1]^{T \times m}$, and $\mathbf{I} = \{\mathbf{1} - \alpha_i\}_{i=1}^T \in [0, 1]^{T \times m}$. The algorithm demonstrates that GSA can be modeled as a two-pass GLA, as illustrated in Fig. 5.

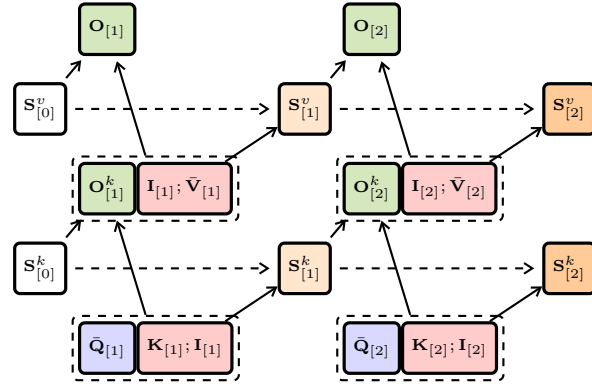


Figure 5: Diagrams of the recurrence and updating rules in Gated Slot Attention. The outputs of the first pass is taken as queries of the second pass.

Legend:
■: query nodes
■: key/value nodes
■: output nodes
■: recurrent hidden states

In the preprocessing step, we precompute the chunkwise cumulative sum of the forget gate, resulting in $\vec{\mathcal{A}}$. Subsequently, $\vec{\mathcal{A}}$ along with the queries, keys, and values are passed to engage in two GLA passes. For each chunk of size C , we define $\overleftarrow{\mathcal{A}}_{[i]} := \vec{\mathcal{A}}_{[i],C} / \vec{\mathcal{A}}_{[i]}$ as in Eq. 15 and Eq. 16.

In the first pass, $\vec{\mathcal{A}}, \overleftarrow{\mathcal{A}}$ is absorbed into $\mathbf{Q}, \mathbf{K}$: $\bar{\mathbf{Q}}_{[i]} = \mathbf{Q}_{[i]} \odot \vec{\mathcal{A}}_{[i]}$, $\bar{\mathbf{K}}_{[i]} = \mathbf{K}_{[i]} \odot (\overleftarrow{\mathcal{A}}_{[i]} / \vec{\mathcal{A}}_{[i],C})$, then $\bar{\mathbf{Q}}$ and $\bar{\mathbf{K}}$ function as usual queries and keys, and the slot representations $\mathbf{I}$ serve as the value vectors.

$$\mathbf{O}_{[i]}^k = \underbrace{\bar{\mathbf{Q}}_{[i]} \mathbf{S}_{[i-1]}^k}_{\mathbf{O}_{[i]}^{\text{inter}}} + \underbrace{((\bar{\mathbf{Q}}_{[i]} \bar{\mathbf{K}}_{[i]}^\top \odot \mathbf{M}) \mathbf{I}_{[i]})}_{\mathbf{O}_{[i]}^{\text{intra}}} \in \mathbb{R}^{C \times m}$$

We use different notations from those presented in Eq.6 to enhance clarity in the chunkwise updating rules. The output $\mathbf{O}^k$ is decomposed into the *inter-chunk* recurrence and *intra-chunk* parallel computations [96].

Algorithm 1 Hardware-Efficient Gated Slot Attention

```

Define FORWARDPASS(Q, K, V, I, A)
  Divide Q, K, V  $\in \mathbb{R}^{T \times d}$ , I, A  $\in \mathbb{R}^{T \times m}$ 
  into  $N = \lceil \frac{T}{C} \rceil$  blocks  $\triangleright C$  is chunk size
  function chunk_cumsum(A)
    parfor  $n \leftarrow 1, N$  do
      Load A[ $n$ ] to SRAM
      Store  $\vec{\mathcal{A}}_{[n]} \leftarrow \text{cumsum}(\mathbf{A}_{[n]})$  to HBM
    return  $\vec{\mathcal{A}} \leftarrow \vec{\mathcal{A}}_{[0]}, \dots, \vec{\mathcal{A}}_{[N]}$ 

  function gsa_fwd(Q, K, V,  $\vec{\mathcal{A}}$ , GATE_K)
    On chip: construct causal mask M  $\in \mathbb{R}^{C \times C}$ 
    for  $n \leftarrow 1, N$  do
      Store S to HBM as S[ $n$ ]  $\triangleright$  Initialize S = 0
      Load K[ $n$ ], V[ $n$ ],  $\vec{\mathcal{A}}_{[n],C}$ ,  $\vec{\mathcal{A}}_{[n]}$  to SRAM
      On chip:  $\vec{\mathcal{A}}_{[n]} \leftarrow \vec{\mathcal{A}}_{[n],C} / \vec{\mathcal{A}}_{[n]}$ 
      if GATE_K then
        S  $\leftarrow \text{Diag}(\vec{\mathcal{A}}_{[n],C})\mathbf{S} + (\mathbf{K}_{[n]} \odot \vec{\mathcal{A}}_{[n]})^\top \mathbf{V}_{[n]}$ 
      else
        S  $\leftarrow \text{SDiag}(\vec{\mathcal{A}}_{[n],C}) + \mathbf{K}_{[n]}^\top (\mathbf{V}_{[n]} \odot \vec{\mathcal{A}}_{[n]})$ 
      parfor  $n \leftarrow 1, N$  do
        Load Q[ $n$ ], K[ $n$ ], V[ $n$ ], S[ $n$ ],  $\vec{\mathcal{A}}_{[n]}$ ,  $\vec{\mathcal{A}}_{[n]}$  to SRAM
        On chip:
          if GATE_K then
             $\bar{\mathbf{Q}}_{[n]} \leftarrow \mathbf{Q}_{[n]} \odot \vec{\mathcal{A}}_{[n]}$ 
             $\bar{\mathbf{K}}_{[n]} \leftarrow \mathbf{K}_{[n]} \odot (\vec{\mathcal{A}}_{[n]} / \vec{\mathcal{A}}_{[n],C})$ 
             $\mathbf{O}_{[n]} \leftarrow \bar{\mathbf{Q}}_{[n]} \mathbf{S}_{[n-1]} + (\mathbf{P} \equiv \bar{\mathbf{Q}}_{[n]} \bar{\mathbf{K}}_{[n]}^\top \odot \mathbf{M}) \mathbf{V}_{[n]}$ 
          else
             $\bar{\mathbf{V}}_{[n]} \leftarrow \mathbf{V}_{[n]} \odot (\vec{\mathcal{A}}_{[n]} / \vec{\mathcal{A}}_{[n],C})$ 
             $\mathbf{O}_{[n]} \leftarrow \bar{\mathbf{Q}}_{[n]} \mathbf{S}_{[n-1]} + (\mathbf{P} \equiv \bar{\mathbf{Q}}_{[n]} \mathbf{K}_{[n]}^\top \odot \mathbf{M}) \bar{\mathbf{V}}_{[n]}$ 
             $\mathbf{O}_{[n]} \leftarrow \mathbf{O}_{[n]} \odot \vec{\mathcal{A}}_{[n]}$ 
        Store O to HBM as O[ $n$ ]
      return O[1,...,N], S[1,...,N]

   $\vec{\mathcal{A}} \leftarrow \text{chunk\_cumsum}(\mathbf{A})$   $\triangleright$  preprocessing
   $\mathbf{O}^k, \mathbf{S}^k \leftarrow \text{gsa\_fwd}(\mathbf{Q}, \mathbf{K}, \mathbf{I}, \vec{\mathcal{A}}, \text{False})$ 
   $\mathbf{Q}^v \leftarrow \text{softmax}(\mathbf{O}^k)$ 
   $\mathbf{O}, \mathbf{S}^v \leftarrow \text{gsa\_fwd}(\mathbf{Q}^v, \mathbf{I}, \mathbf{V}, \vec{\mathcal{A}}, \text{True})$ 
  return O

Define BACKWARDPASS(Q, K, V, I, A,  $\mathbf{O}^k$ , dO)
  Divide Q, K, V, O, dO  $\in \mathbb{R}^{T \times d}$ , I, A  $\in \mathbb{R}^{T \times m}$ 
  into  $N = \lceil \frac{T}{C} \rceil$  blocks  $\triangleright C$  is chunk size
  function gsa_bwd(Q, K, V, S,  $\vec{\mathcal{A}}$ , dO, GATE_K)
    On chip: construct causal mask M  $\in \mathbb{R}^{C \times C}$ 
    for  $n \leftarrow N, 1$  do  $\triangleright$  in reverse order
      Store dS in HBM as dS[ $n$ ]  $\triangleright$  Initialize dS = 0
      Load Q[ $n$ ],  $\vec{\mathcal{A}}_{[n]}$ , dO[ $n$ ] to SRAM
      On chip:
        if GATE_K then
          dS  $\leftarrow \text{Diag}(\vec{\mathcal{A}}_{[n],C})\mathbf{dS} + (\mathbf{Q}_{[n]} \odot \vec{\mathcal{A}}_{[n]})^\top \mathbf{dO}_{[n]}$ 
        else
          dS  $\leftarrow \text{dSDiag}(\vec{\mathcal{A}}_{[n],C}) + \mathbf{Q}_{[n]}^\top (\mathbf{dO}_{[n]} \odot \vec{\mathcal{A}}_{[n]})$ 
      parfor  $n \leftarrow 1, N$  do
        Load Q[ $n$ ], K[ $n$ ], V[ $n$ ], dO[ $n$ ]  $\in \mathbb{R}^{C \times d}$ 
        S[ $n$ ], dS[ $n$ ]  $\in \mathbb{R}^{d \times d}$  to SRAM
        On chip:  $\triangleright$  Recompute  $\vec{\mathcal{A}}_{[n]}$ ,  $\bar{\mathbf{Q}}_{[n]}$ ,  $\bar{\mathbf{K}}_{[n]}$ ,  $\bar{\mathbf{V}}_{[n]}$ , P
        if GATE_K then
          dP  $\leftarrow (\mathbf{dO}_{[n]} \mathbf{V}_{[n]}^\top) \odot \mathbf{M}$ 
          dQ  $\leftarrow (\mathbf{dO}_{[n]} \mathbf{S} + \mathbf{dP} \bar{\mathbf{K}}_{[n]}^\top) \odot \vec{\mathcal{A}}_{[n]}$ 
          dK  $\leftarrow (\mathbf{V}_{[n]} \mathbf{dS}^\top + \mathbf{dP}^\top \bar{\mathbf{Q}}_{[n]}) \odot \vec{\mathcal{A}}_{[n]}$ 
          dV  $\leftarrow \bar{\mathbf{K}}_{[n]} \mathbf{dS}_{[n]} + \mathbf{P}^\top \mathbf{dO}_{[n]}$ 
        else
          dP  $\leftarrow (\mathbf{dO}_{[n]} \bar{\mathbf{V}}_{[n]}^\top) \odot \mathbf{M}$ 
          dQ  $\leftarrow \mathbf{dO}_{[n]} \mathbf{S}^\top + \mathbf{dP} \mathbf{K}_{[n]}$ 
          dK  $\leftarrow \bar{\mathbf{V}}_{[n]} \mathbf{dS}^\top + \mathbf{dP}^\top \mathbf{Q}_{[n]}$ 
          dV  $\leftarrow (\mathbf{K}_{[n]} \mathbf{dS}_{[n]} + \mathbf{P}^\top \mathbf{dO}_{[n]}) \odot \vec{\mathcal{A}}_{[n]}$ 
        Write dQ, dK, dV to HBM as dQ[ $n$ ], dK[ $n$ ], dV[ $n$ ]
      return dQ[1,...,N], dK[1,...,N], dV[1,...,N]

    Recompute  $\vec{\mathcal{A}}, \mathbf{S}^k, \mathbf{S}^v$ 
    dQ $v$ , dI $v$ , dV  $\leftarrow \text{gsa\_bwd}(\mathbf{Q}, \mathbf{I}, \mathbf{V}, \mathbf{S}^v, \vec{\mathcal{A}}, \mathbf{dO}, \text{False})$ 
    dO $k$   $\leftarrow \text{dsoftmax}(\mathbf{O}^k, \mathbf{dQ}^v)$   $\triangleright$  softmax gradients
    dQ, dK, dI $k$   $\leftarrow \text{gsa\_bwd}(\mathbf{Q}, \mathbf{K}, \mathbf{I}, \mathbf{S}^k, \vec{\mathcal{A}}, \mathbf{dO}^k, \text{True})$ 
    dI  $\leftarrow \mathbf{dI}^k + \mathbf{dI}^v$ 
    dA  $\leftarrow \text{reversed\_cumsum}(\mathbf{Q} \odot \mathbf{dQ} - \mathbf{K} \odot \mathbf{dK} + \mathbf{O} \odot \mathbf{dO} - \mathbf{V} \odot \mathbf{dV})$ 
    return dQ, dK, dV, dI, dA
  
```

In the second pass, the output $\mathbf{O}^k$ from the first pass, after the application of the softmax function, serves as the queries $\mathbf{Q}^v$,

$$\mathbf{Q}^v = \text{softmax}(\mathbf{O}^k)$$

and **I/V** are used as the key/value vectors, respectively. The final GSA output **O** is obtained as follows:

$$\mathbf{O}_{[i]} = \mathbf{Q}_{[i]}^v \mathbf{S}_{[i-1]}^v + ((\mathbf{Q}_{[i]}^v \mathbf{I}_{[i]}^\top) \odot \mathbf{M}) \bar{\mathbf{V}}_{[i]} \in \mathbb{R}^{C \times d}$$

Unlike in the first pass, $\vec{\mathcal{A}}, \vec{\mathcal{A}}$ is absorbed into **V**, **O** rather than **Q**, **K**.

During the backward pass, computing the gradients of **Q**, **K**, **V**, **I**, **A** involves variables already computed in the forward pass. However, directly saving all intermediate results can pose severe challenges for memory management. To address this issue, we adopt gradient checkpointing [11] to trade off memory consumption for recomputation. In addition to the input **Q**, **K**, **V**, **I**, **A**, we selectively save only the output of the first GLA pass, which significantly reduces memory consumption (Figure 4b).

Similar to the forward pass, the backward pass involves two GLA backward passes as well, but in the reverse order. The final gradient **dI** is obtained by combining the gradients from these computations, i.e., **dI** = **dI** ^{k} + **dI** ^{v} . The forget gate gradient can be decomposed into two parts: $\mathbf{Q} \odot \mathbf{dQ} - \mathbf{K} \odot \mathbf{dK}$ and $\mathbf{O} \odot \mathbf{dO} - \mathbf{V} \odot \mathbf{dV}$ (cf. §C in [96]). The reversed cumulative sum in the backward pass corresponds to the cumulative sum computed in the preprocessing step of the forward pass.

We provide a PyTorch implementation for the above algorithm with chunkwise parallelism in Listing 1.

```

1 def gsa_fwd_k(q, k, v, g, C):
2     '''
3     q/k/v:
4         query, key, value of shape [NC, C, K|V]
5     g:
6         local cumulative product of forget gate in log space
7     C:
8         chunk size
9     '''
10    # NC: number of chunks
11    # K: query/key head dimension
12    # V: value head dimension
13    NC, C, K, V = *q.shape, v.shape[-1]
14    # [K, V]
15    s = q.new_zeros(K, V)
16    # [NC, C, V]
17    o = torch.empty_like(v)
18
19    for i in range(0, NC):
20        # [C, K|V] chunking
21        c_q, c_k, c_v, c_g = q[i], k[i], v[i], g[i]
22        # the last g of each chunk
23        c_gn = c_g[-1]
24        # inter-chunk w/ matmul
25        c_vg, c_gn = c_v * (c_gn - c_g).exp(), c_gn.exp()
26        # [C, V]
27        c_o_inter = (c_q @ s) * c_g.exp()
28        # hidden state update
29        s = c_gn * s + c_k.t() @ c_vg
30
31        # intra-chunk
32        # [C, C]
33        c_A = c_q @ c_k.t()
34        # [C, V]
35        c_o_intra = torch.zeros_like(c_v)
36        for j in range(0, C // 16):
37            t = slice(j * 16, j * 16 + 16)
38            # [16, K|V] subchunking
39            s_A, s_v, s_g = c_A[t], c_v[t], c_g[t]
40            s_o = q.new_zeros(16, V)
41
42            # inter-subchunk w/ matmul
43            s_gn = s_g[0]
44            for si in range(0, j):
45                u = slice(si * 16, si * 16 + 16)
46                s_o += s_A[:, u] @ (c_v[u] * (s_gn - c_g[u]).exp())
47            s_o *= (s_g - s_gn).exp()
48            # intra-subchunk w/o matmul
49            for si in range(16):
50                for sj in range(si + 1):
51                    s_o[si] += s_A[si, j * 16 + sj] * s_v[sj] * (s_g[si] - s_g[sj]).exp()
52            c_o_intra[t] = s_o
53        # [C, V]
54        o[i] = c_o_inter + c_o_intra
55    return o
56
57
58 def gsa_fwd_v(q, k, v, g, C):
59    NC, C, K, V = *q.shape, v.shape[-1]
60    s = q.new_zeros(K, V)
61    o = torch.empty_like(v)
62
63    for i in range(0, NC):

```

```

64     # [C, K|V] chunking
65     c_q, c_k, c_v, c_g = q[i], k[i], v[i], g[i]
66     # the last g of each chunk
67     c_gn = c_g[-1]
68     # inter-chunk w/ matmul
69     c_qg, c_kg, c_gn = c_q * c_g.exp(), c_k * (c_gn - c_g).exp(), c_gn.exp()
70     # [C, V]
71     c_o_inter = c_qg @ s
72     # hidden state update
73     s = c_gn[:, None] * s + c_kg.t() @ c_v
74
75     # intra-chunk
76     c_A = q.new_zeros(C, C)
77     for j in range(0, C // 16):
78         t = slice(j * 16, j * 16 + 16)
79         # [16, K|V] subchunking
80         s_q, s_k, s_g = c_q[t], c_k[t], c_g[t]
81         s_A = q.new_zeros(16, 16)
82
83         # intra-subchunk w/o matmul
84         for si in range(16):
85             for sj in range(si + 1):
86                 s_A[si, sj] = torch.sum(s_q[si] * s_k[sj] * (s_g[si] - s_g[sj]).exp())
87         c_A[t, t] = s_A
88         # inter-subchunk w/ matmul
89         s_gn = s_g[0]
90         s_qg = s_q * (s_g - s_gn).exp()
91         for si in range(0, j):
92             u = slice(si * 16, si * 16 + 16)
93             c_A[t, u] = s_qg @ (c_k[u] * (s_gn - c_g[u]).exp()).t()
94         c_o_intra = c_A @ c_v
95         # [C, V]
96         o[i] = c_o_inter + c_o_intra
97     return o
98
99
100 def gsa(q, k, v, s, g):
101     T, M = s.shape
102     # reshape each input to [NC, C, K|V]
103     q, k, v, s, g = map(lambda x: x.view(-1, C, x.shape[-1]), (q, k, v, s, g))
104     # local compute of cumulative product of decay
105     # [NC, C, K]
106     g = g.cumsum(1)
107     ok = gsa_fwd_k(q, k, s, g, M)
108     qv = ok.softmax(-1)
109     o = gsa_fwd_v(qv, s, v, g, M)
110     return o.view(T, -1)

```

Listing 1: Pseudo PyTorch-style code snippet for GSA with chunkwise parallelism. For brevity, we omit the dimensions of batch size and number of heads. Notably, unlike Algorithm 1, we obtain the intra outputs via a secondary chunking strategy in Line 31-52 and Line 75-94, as utilized by GLA [96], to ensure numerical stability.

C Experimental Setup

C.1 Language Modeling

We compare GSA with the following strong Transformers with modern architectural recipes as well as other recent subquadratic architectures:

- Xfmr++ [86]: Llama-like architectures that enhance the vanilla Transformer by using Rotary position embeddings [80] and GLU [78].

- Mamba [29]: State-space models with *data-dependent* decay.
- RetNet [82]: Linear attention with non-learnable, *data-independent head-wise* decay and rotary embedding.
- GLA [96]: Linear attention with elementwise *data-dependent* decay.
- HGRN2 [69]: Gated Linear RNN with state expansion, or GLA with improved parameterization.

Setup. For a fair comparison, all models are trained from scratch with the same training recipes. We utilize a subset of 100B tokens picked from the Slimpajama dataset [79]. The input tokens are processed using the Mistral tokenizer [39]¹¹. We use AdamW [50] with a weight decay 0.01 as the optimizer. During training, the learning rate is first warmed up to 3×10^{-4} in the first 1B tokens, and then decayed to 3×10^{-5} gradually with a cosine schedule. The number of attention heads is set to 4 and 5 for 1.3B and 2.7B models, respectively. The number of memory slots is uniformly set to 64 for all models. We utilize the open-sourced Triton-based library FLA [95] to run all compared models.

We ran all models on 32 Nvidia H800 GPUs. To facilitate distributed training and accelerate the process, we utilized the DeepSpeed framework and fused all necessary modules, including ROPE, cross-entropy, and LayerNorm, following the practice of [102]. The training of a GSA model with 2.7B parameters took approximately 2 days, while the 1.3B model required 1 day to complete training.

Remark on state size. Let the model dimension be denoted as d . Mamba expands the value projection to $2d$ and uses a state expansion ratio of 16, resulting in a state size of $32d$ per layer. Since Mamba also replaces the FFN with a Mamba layer, this effectively doubles both the number of recurrent layers and the state size, leading to a total recurrent state size of $64Ld$.

Similarly, RetNet expands the value projection to $2d$ and sets the head dimension of queries/keys to be half that of the value head dimension. RetNet also reduces the number of heads to increase the head dimensions of queries and keys. We fix the query/key head dimension to 256 and adjust the number of heads accordingly, resulting in a recurrent state size of $512d$ per layer and $512Ld$ in total.

GLA does not expand the value projection but reduces the head dimensions of queries and keys to half of the value head dimension to save parameters for the Swish output gate, ensuring each layer contains $4d^2$ parameters. We fix the query/key head dimension to 256 and adjust the number of heads accordingly, resulting in a recurrent state size of $256d$ per layer and $256Ld$ in total.

HGRN2 follows a similar approach to GLA but without the Swish output gate, keeping the head dimensions of queries/keys and values equal, as in standard softmax attention, while still retaining $4d^2$ total parameters per recurrent layer. We set the head dimension to 128, resulting in a recurrent state size of $128d$ per layer and $128Ld$ in total.

GSA maintains hidden states for both keys and values, so each layer contains a recurrent state size of $2 \times 64 \times d$. We fix the state expansion (i.e., number of slots) to 64^{12} , resulting in a total recurrent state size of $128Ld$.

¹¹<https://huggingface.co/mistralai/Mistral-7B-v0.1>

¹²Note that in this case, the number of heads is independent of the state expansion ratio